Advanced Functions

1. Window Functions

- A way to combine Group By aggregation with regular select statements.
- The value of aggregation is calculated without combining the returned rows.
- Syntax
  SELECT * function OVER (PARTITION BY field_name) FROM …

```
function_name(column) OVER (PARTITION BY column ORDER BY column)
```

1. Window Functions

1. ROW_NUMBER()
Definition
Assigns a **unique sequential number** to each row.

The ROW_NUMBER() window function in SQL assigns a unique, sequential integer to each row within a result set or within specific partitions of data. It is primarily used for **ranking**, **filtering**, **de-duplication**, and **pagination**.

```sql
SELECT name, salary,
ROW_NUMBER( ) OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

name character varying (50)	salary numeric	row_num bigint
Name	14000	1
Name	13000	2
Name	13000	3
Mostafa	12000	4
Hassan	10000	5
Mona	9000	6
Ahmed	8000	7
Nour	7500	8
Mai	7500	9
mohAMed	7100	10
Mohamed	7000	11

1. Window Functions

2. RANK()
Definition
Ranks rows, but **same values get the same rank**, and ranks **skip numbers.**

The RANK() function in SQL is a window function used to assign a ranking number to each row within a partition of a result set. If two or more rows tie (have identical values in the ordering column), they receive the same rank, and RANK() will skip the next consecutive rank(s), leaving gaps in the sequence.

```
SELECT name, salary,
RANK( ) OVER (ORDER BY salary DESC) AS rank
FROM employees;
```

name character varying (50)	salary numeric	row_num bigint
Name	14000	1
Name	13000	2
Name	13000	2
Mostafa	12000	4
Hassan	10000	5
Mona	9000	6
Ahmed	8000	7
Nour	7500	8
Mai	7500	8
mohAMed	7100	10
Mohamed	7000	11

rows: 19 | Query complete 00:00:00.193

1. Window Functions

3. DENSE_RANK()
Definition
Similar to RANK() but **does not skip numbers.**

DENSE_RANK() is a SQL window function that assigns a rank to each row within a result set partition, where rows with equal values receive the same rank and, unlike RANK(), **no ranks are skipped** in the sequence.

```
SELECT name, salary,
DENSE_RANK( ) OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

name character varying (50)	salary numeric	row_num bigint
Name	14000	1
Name	13000	2
Name	13000	2
Mostafa	12000	3
Hassan	10000	4
Mona	9000	5
Ahmed	8000	6
Nour	7500	7
Mai	7500	7
mohAMed	7100	8
Mohamed	7000	9

1. Window Functions

4. NTILE()
Definition
Divides rows into **equal groups (buckets)**.

The **NTILE()** window function in SQL divides an ordered result set into a specified number of approximately equal-sized groups, or "buckets," and assigns a ranking number to each row based on its bucket. It is essential for data segmentation, ranking, and identifying trends.

```sql
SELECT name, salary,
NTILE(2) OVER (ORDER BY salary DESC) AS group_number
FROM employees;
```

name character varying (50)	salary numeric	group_number integer
Name	14000	1
Name	13000	1
Name	13000	1
Mostafa	12000	1
Hassan	10000	1
Mona	9000	1
Ahmed	8000	1
Nour	7500	1
Mai	7500	1
mohAMed	7100	1
Mohamed	7000	2

rows: 19 Query complete 00:00:00.212

Meaning:
Divide employees into **2 groups**

1. Window Functions

5. LAG()
Definition
Gets the **previous row value.**

The LAG function in SQL is a window function that accesses data from a previous row in the same result set without using complex self-joins. It is essential for time-series analysis and comparing current rows with historical ones.

```
SELECT name, salary,
LAG(salary) OVER (ORDER BY salary) AS previous_salary
FROM employees;
```

name character varying (50)	salary numeric	prev_salary numeric
Name	14000	[null]
Name	13000	14000
Name	13000	13000
Mostafa	12000	13000
Hassan	10000	12000
Mona	9000	10000
Ahmed	8000	9000
Nour	7500	8000
Mai	7500	7500
mohAMed	7100	7500
Mohamed	7000	7100

rows: 19 Query complete 00:00:00.269

1. Window Functions

6. LEAD()
Definition
Gets the **next row value.**

The LEAD() window function in SQL is a powerful tool designed to look ahead, allowing you to access data from subsequent rows within the same result set without using complex self-joins or subqueries. It is particularly effective for time-series data, sequence analysis, and comparative analytics.

```sql
SELECT name, salary,
LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM employees;
```

name character varying (50)	salary numeric	next_salary numeric
Name	14000	13000
Name	13000	13000
Name	13000	12000
Mostafa	12000	10000
Hassan	10000	9000
Mona	9000	8000
Ahmed	8000	7500
Nour	7500	7500
Mai	7500	7100
mohAMed	7100	7000
Mohamed	7000	6000

rows: 19 Query complete 00:00:00.236

1. Window Functions

7. SUM() as Window Function
Definition
Calculates **running totals.**

```
SELECT name, salary,
SUM(salary) OVER (ORDER BY salary) AS running_total
FROM employees;
```

name character varying (50)	salary numeric	running_total numeric
Name	14000	14000
Name	13000	40000
Name	13000	40000
Mostafa	12000	52000
Hassan	10000	62000
Mona	9000	71000
Ahmed	8000	79000
Nour	7500	94000
Mai	7500	94000
mohAMed	7100	101100
Mohamed	7000	108100

rows: 19 | Query complete 00:00:00.184

1. Window Functions

8. AVG() as Window Function
Definition
Calculates average across the window.

```
SELECT name, salary,
AVG(salary) OVER () AS avg_salary
FROM employees;
```

name character varying (50)	salary numeric	avg_salary numeric
Name	13000	7278.9473684210526316
Name	14000	7278.9473684210526316
Mohamed	7000	7278.9473684210526316
Mona	9000	7278.9473684210526316
Ali	5000	7278.9473684210526316
mohAMed	7100	7278.9473684210526316
Ahmed	8000	7278.9473684210526316
Sara	6000	7278.9473684210526316
Try Transaction	1000	7278.9473684210526316
Try Transaction	1000	7278.9473684210526316
Nour	7500	7278.9473684210526316

1. Window Functions

ORDER BY in the OVER clause

```
SELECT salary, sum(salary) OVER (ORDER BY salary) FROM
```

1. Window Functions

Summary

Function	Purpose
ROW_NUMBER()	unique row number
RANK()	ranking with gaps
DENSE_RANK()	ranking without gaps
NTILE()	divide rows into groups
LAG()	previous row value
LEAD()	next row value
SUM() OVER()	running total
AVG() OVER()	window average

2. ARRAY_AGG - Aggregate into Array

ARRAY_AGG() is an **aggregate function** that **collects values from multiple rows and returns them as an array.**
Instead of returning many rows, it **combines them into one array.**

```
SELECT ARRAY_AGG(name) AS employee_names
FROM employees;
```

array_agg character varying[]
{Hassan,Mai,Mostafa,Amr,sAlma,Alex,Name,Name,Name,Name,Mohamed,Mona,Ali,mohAMed,Ahmed,Sara,"Try Transaction","Try Transaction",...

2. ARRAY_AGG - Aggregate into Array

ARRAY_AGG with GROUP BY

```
SELECT dep_id, ARRAY_AGG(name) AS employees
FROM employees
GROUP BY dep_id;
```

	array_agg character varying[]
3	{Hassan,Mai,sAlma,Alex,Name,Name,Name,Mona,"Try Transaction","Try Transacti...
4	{Nour}
2	{Mostafa,Name,Mohamed,Ali,mohAMed}
1	{Amr,Ahmed,Sara}

2. ARRAY_AGG - Aggregate into Array

3. ARRAY_AGG with ORDER BY

```
SELECT ARRAY_AGG(name ORDER BY salary DESC) AS employees_by_salary
FROM employees;
```

array_agg character varying[]
{Name,Name,Name,Mostafa,Hassan,Mona,Ahmed,Nour,Mai,mohAMed,Mohamed,Sara,Alex,sAlma,Ali,Amr,Name,"Try Transaction","Try Transac...

2. ARRAY_AGG - Aggregate into Array

4. ARRAY_AGG with DISTINCT

```
SELECT ARRAY_AGG(DISTINCT dep_id) AS departments
FROM employees;
```

array_agg integer[]
{1,2,3,4}

3. BOOL_AND / BOOL_OR

BOOL_AND() and BOOL_OR() are **aggregate functions for boolean values**. They evaluate a **set of TRUE/FALSE values** and return:

Function	Result
`BOOL_AND()`	TRUE if all values are TRUE, otherwise FALSE
`BOOL_OR()`	TRUE if any value is TRUE, otherwise FALSE

```sql
SELECT dep_id,
       BOOL_AND(salary > 4000) AS all_above_4000
FROM employees
GROUP BY dep_id;
```

dep_id integer	all_above_4000 boolean
3	false
4	true
2	true
1	false

```sql
SELECT dep_id,
       BOOL_OR(salary > 7500) AS any_above_7500
FROM employees
GROUP BY dep_id;
```

dep_id integer	all_above_5000 boolean
3	true
4	false
2	true
1	true

5. JOIN WITH WINDOW FUN

First, We will add in enrollment table Score Called

```
ALTER TABLE enrollments ADD score float DEFAULT 0;
select * from enrollments;
update enrollments set score=60 where enrollment_id=1;
update enrollments set score=80 where enrollment_id=2;
update enrollments set score=40 where enrollment_id=3;
update enrollments set score=75 where enrollment_id=4;
select * from enrollments;
```

5. JOIN WITH WINDOW FUN

Rank Students by Score Within Each Course

```sql
SELECT
    s.student_id,
    s.student_name,
    c.course_name,
    e.score,
    RANK( ) OVER (
        PARTITION BY c.course_id
        ORDER BY e.score DESC
    ) AS rank_in_course
FROM enrollments e
JOIN students s
    ON e.student_id = s.student_id
JOIN courses c
    ON e.course_id = c.course_id;
```

student_id integer	student_name character varying (50)	course_name character varying (50)	score double precision	rank_in_course bigint
1	Ali	Database	60	1
2	Sara	Database	40	2
1	Ali	JS	80	1
3	Omar	HTML	75	1

- Joins students, courses, and enrollments
  - Ranks students **within each course**
  - Highest score = rank 1

5. JOIN WITH WINDOW FUN

Task :

In database courses : Get Top Student Per Course-> Returns **highest scoring student per course.**
In database company : **Return the highest paid employee in each department.**

5. JOIN WITH WINDOW FUN

Get Top Student Per Course

```
SELECT *
FROM (
    SELECT
        s.student_id,
        s.student_name,
        c.course_name,
        e.score,
        ROW_NUMBER( ) OVER (
            PARTITION BY c.course_id
            ORDER BY e.score DESC
        ) AS rn
    FROM enrollments e
    JOIN students s
        ON e.student_id = s.student_id
    JOIN courses c
        ON e.course_id = c.course_id
) t
WHERE rn = 1;
```

student_id integer	student_name character varying (50)	course_name character varying (50)	score double precision	rn bigint
1	Ali	Database	60	1
1	Ali	JS	80	1
3	Omar	HTML	75	1

Returns **highest scoring student per course.**

5. JOIN WITH WINDOW FUN

Number of Parents per Student

```sql
SELECT
    s.student_id,
    s.student_name,
    p.name,
    COUNT(p.id) OVER (
        PARTITION BY s.student_id
    ) AS total_parents
FROM students s
JOIN student_parents sp
    ON s.student_id = sp.student_id
JOIN parents p
    ON p.id = sp.parent_id;
```

student_id integer	student_name character varying (50)	name text	total_parents bigint
1	Ali	Ahmed Hass...	2
1	Ali	Fatima Ali	2
2	Sara	Ahmed Hass...	1
3	Omar	Mohamed Sa...	1
4	Ahmed	Sara Ibrahim	2
4	Ahmed	Omar Khaled	2
5	Karim	Mohamed Sa...	1

Each row shows the parent but also **how many parents the student has.**

5. JOIN WITH WINDOW FUN

Show Student Rank Across All Courses

```
SELECT
    s.student_id,
    s.student_name,
    c.course_name,
    e.score,
    DENSE_RANK( ) OVER (
        ORDER BY e.score DESC
    ) AS global_rank
FROM enrollments e
JOIN students s
    ON s.student_id = e.student_id
JOIN courses c
    ON c.course_id = e.course_id;
```

student_id integer	student_name character varying (50)	course_name character varying (50)	score double precision	global_rank bigint
1	Ali	JS	80	1
3	Omar	HTML	75	2
1	Ali	Database	60	3
2	Sara	Database	40	4

Ranks **all scores globally.**

Thank you